

Then the short-circuiting effect of `&&` would only prevent `B()` from being evaluated, but because `A() && B()` as a whole is `false`, `C()` would still be evaluated.

However, note that short-circuiting does not change the final evaluation outcome. It only affects the evaluation of *operands*, not how *operators* are grouped — if evaluation of operands doesn't have side effects (for example, logging to the console, assigning to variables, throwing an error), short-circuiting would not be observable at all.

The assignment counterparts of these operators (`&&=`, `||=`, `??=`) are short-circuited as well. They are short-circuited in a way that the assignment does not happen at all.

Table

The following table lists operators in order from highest precedence (18) to lowest precedence (1).

Several general notes about the table:

1. Not all syntax included here are "operators" in the strict sense. For example, spread `...` and arrow `=>` are typically not regarded as operators. However, we still included them to show how tightly they bind compared to other operators/expressions.
2. Some operators have certain operands that require expressions narrower than those produced by higher-precedence operators. For example, the right-hand side of member access `.` (precedence 17) must be an identifier instead of a grouped expression. The left-hand side of arrow `=>` (precedence 2) must be an arguments list or a single identifier instead of some random expression.
3. Some operators have certain operands that accept expressions wider than those produced by higher-precedence operators. For example, the bracket-enclosed expression of bracket notation `[ ... ]` (precedence 17) can be any expression, even comma (precedence 1) joined ones. These operators act as if that operand is "automatically grouped". In this case we will omit the associativity.

Precedence	Associativity	Individual operators	Notes
18: grouping	n/a	Grouping <code>(x)</code>	[1]
17: access and call	left-to-right	Member access <code>x.y</code>	[2]
		Optional chaining <code>x?.y</code>	
	n/a	Computed member access <code>x[y]</code>	[3]
		new with argument list <code>new x(y)</code>	[4]
		Function call <code>x(y)</code>	
16: new	n/a	import(x) new without argument list <code>new x</code>	

15: postfix operators	n/a	Postfix increment <code>x++</code>	[5]
		Postfix decrement <code>x--</code>	
14: prefix operators	n/a	Prefix increment <code>++x</code>	[6]
		Prefix decrement <code>--x</code>	
		Logical NOT <code>!x</code>	
		Bitwise NOT <code>~x</code>	
		Unary plus <code>+x</code>	
		Unary negation <code>-x</code>	
		<code>typeof x</code>	
		<code>void x</code>	
		<code>delete x</code>	[7]
		<code>await x</code>	
13: exponentiation	right-to-left	Exponentiation <code>x ** y</code>	[8]
12: multiplicative operators	left-to-right	Multiplication <code>x * y</code>	
		Division <code>x / y</code>	
		Remainder <code>x % y</code>	
11: additive operators	left-to-right	Addition <code>x + y</code>	
		Subtraction <code>x - y</code>	
10: bitwise shift	left-to-right	Left shift <code>x << y</code>	

9: relational operators	left-to-right	Right shift <code>x >> y</code>	
		Unsigned right shift <code>x >>> y</code>	
		Less than <code>x < y</code>	
		Less than or equal <code>x <= y</code>	
		Greater than <code>x > y</code>	
		Greater than or equal <code>x >= y</code>	
8: equality operators	left-to-right	<code>x in y</code>	
		<code>x instanceof y</code>	
		Equality <code>x == y</code>	
		Inequality <code>x != y</code>	
7: bitwise AND	left-to-right	Strict equality <code>x === y</code>	
		Strict inequality <code>x !== y</code>	
6: bitwise XOR	left-to-right	Bitwise AND <code>x & y</code>	
5: bitwise OR	left-to-right	Bitwise XOR <code>x ^ y</code>	
4: logical AND	left-to-right	Bitwise OR <code>x y</code>	
3: logical OR, nullish coalescing	left-to-right	Logical AND <code>x && y</code>	
		Logical OR <code>x y</code>	
		Nullish coalescing operator <code>x ?? y</code>	[9]

2: assignment and miscellaneous	right-to-left	Assignment	<code>x = y</code>	[10]		
		Addition assignment	<code>x += y</code>			
		Subtraction assignment	<code>x -= y</code>			
		Exponentiation assignment	<code>x **= y</code>			
		Multiplication assignment	<code>x *= y</code>			
		Division assignment	<code>x /= y</code>			
		Remainder assignment	<code>x %= y</code>			
		Left shift assignment	<code>x <<= y</code>			
		Right shift assignment	<code>x >>= y</code>			
		Unsigned right shift assignment	<code>x >>>= y</code>			
		Bitwise AND assignment	<code>x &= y</code>			
		Bitwise XOR assignment	<code>x ^= y</code>			
		Bitwise OR assignment	<code>x = y</code>			
		Logical AND assignment	<code>x &&= y</code>			
		Logical OR assignment	<code>x = y</code>			
		Nullish coalescing assignment	<code>x ??= y</code>			
		right-to-left	Conditional (ternary) operator		<code>x ? y : z</code>	[11]

	right-to-left	Arrow <code>x => y</code>	[12]
	n/a	<code>yield x</code>	
		<code>yield* x</code>	
		Spread <code>...x</code>	
1: comma	left-to-right	Comma operator <code>x, y</code>	[13]

Notes:

1. The operand can be any expression.
2. The "right-hand side" must be an identifier.
3. The "right-hand side" can be any expression.
4. The "right-hand side" is a comma-separated list of any expression with precedence > 1 (i.e., not comma expressions). The constructor of a `new` expression cannot be an optional chain.
5. The operand must be a valid assignment target (identifier or property access). Its precedence means `new Foo++` is `(new Foo)++` (a syntax error) and not `new (Foo++)` (a `TypeError: (Foo++) is not a constructor`).
6. The operand must be a valid assignment target (identifier or property access).
7. The operand cannot be an identifier or a `private element` access.
8. The left-hand side cannot have precedence 14.
9. The operands cannot be a logical OR `||` or logical AND `&&` operator without grouping.
10. The "left-hand side" must be a valid assignment target (identifier or property access).
11. The associativity means the two expressions after `?` are implicitly grouped.
12. The "left-hand side" is a single identifier or a parenthesized parameter list.
13. Only valid inside object literals, array literals, or argument lists.

The precedence of groups 17 and 16 may be a bit ambiguous. Here are a few examples to clarify.

- Optional chaining is always substitutable for its respective syntax without optionality (barring a few special cases where optional chaining is forbidden). For example, any place that accepts `a?.b` also accepts `a.b` and vice versa, and similarly for `a?.()`, `a()`, etc.
- Member expressions and computed member expressions are always substitutable for each other.
- Call expressions and `import()` expressions are always substitutable for each other.
- This leaves four classes of expressions: member access, `new` with arguments, function call, and `new` without arguments.
 - The "left-hand side" of a member access can be: a member access (`a.b.c`), `new` with arguments (`new a().b`), and function call (`a().b`).
 - The "left-hand side" of `new` with arguments can be: a member access (`new a.b()`) and `new` with arguments (`new new a()()`).
 - The "left-hand side" of a function call can be: a member access (`a.b()`), `new` with arguments (`new a()()`), and function call (`a()()`).
 - The operand of `new` without arguments can be: a member access (`new a.b`), `new` with arguments (`new new a()`), and `new` without arguments (`new new a`).